

A Phase-Intention Hamiltonian for Relational Dynamics

Aharonov–Bohm, Berry, Euler Closure, the Information/Intention Operator, Collatz
Rhythm, and Free Twin-Prime Seeds

Adrian Lipa / USER007

Formal working note prepared by Zosia / Codex GPT

June 19, 2026

Abstract

This document formulates a working Hamiltonian scaffold in which the free part of the Hamiltonian is not the usual free-particle kinetic term. Instead, the free part is taken to be a phase generator driven by an Information/Intention Operator, scaled by the constant $\ln 2/(24\pi)$, clocked by a Collatz-type discrete rhythm, and seeded by a free twin-prime pair. The horizontal or shape dynamics is then written covariantly with respect to a combined Aharonov–Bohm–Berry–Euler connection. A central Euler–Berry closure constraint is introduced, binding the phase defect ϵ , the topological sector D , and the total holonomy. The document includes derivations, a canonical rotor embedding, a Floquet/discrete-time version, and reference simulations with source code.

Epistemic and technical status

- **Status:** formal ansatz, derivation scaffold, and numerical consistency workbench.
- **Not claimed:** no experimental proof, no external NOEMA CURRENT/ACK, and no canon promotion without a separate receipt.
- **Main structural claim:** the phase sector is treated as the primitive free channel; the usual kinetic/shape term is horizontal dynamics relative to a phase connection.
- **Notation caveat:** D denotes a topological closure sector: degree, winding sector, Chern/Euler sector, or an effective discrete closure index. If the project later fixes a narrower canonical meaning of D , the equations below should be interpreted by substituting that meaning into the same closure role.

Contents

1	Core thesis	3
2	Relational configuration space and phase bundle	3
3	Aharonov–Bohm–Berry–Euler connection	3
4	Information constant, operator domain, and zero-level fluctuations	4
4.1	Meaning of a defined domain	4
5	Twin-prime seed and Collatz/Kolac rhythm	5
6	Euler–Berry constraint binding ϵ , D , and phase	5
6.1	Euler and Berry quantization interpretation	6

7 Derivation I: canonical phase rotor embedding	6
8 Derivation II: discrete Floquet phase generator	7
9 Covariant quantum Hamiltonian	8
10 Consistency conditions	8
11 Reference simulations	8
11.1 Collatz phase accumulation	8
11.2 Euler–Berry closure scan	10
11.3 Finite matrix Hamiltonian check	11
12 Interpretation	12
13 Minimal next formal tasks	12
A Source code: Collatz phase simulation	13
B Source code: Euler–Berry constraint scan	16
C Source code: finite Hamiltonian matrix demo	17
D Source code: run all	19

1 Core thesis

The standard perturbative decomposition

$$H = H_0 + H_{\text{int}} \quad (1)$$

usually takes H_0 to be a free-particle or free-field Hamiltonian. The present relational-phase model uses a different split:

$$H = H_{\Phi, \mathcal{I}, C, s} + H_{\perp}. \quad (2)$$

Here $H_{\Phi, \mathcal{I}, C, s}$ is the free phase/intention sector, while $H_{\perp}$ is the horizontal remainder: shape, amplitude, potential, and deformation dynamics.

The seed is a free twin-prime pair

$$s = (p, p+2), \quad p, p+2 \in \mathbb{P}. \quad (3)$$

The Collatz/Kolac rhythm extracted from s is denoted $\rho_s(k)$, where $k \in \mathbb{N}$ is the discrete orbital step. The Information/Intention Operator is denoted $\hat{\mathcal{I}}_s(\tau, k)$.

The compact working Hamiltonian is

$$\hat{H}_s(\tau, k) = \frac{\hbar}{\Delta\tau_k} \rho_s(k) \left[\frac{\ln 2}{24\pi} \hat{W}_s + \delta \hat{\mathcal{I}}_0(\tau) \right] + \frac{1}{2} g^{ab} \hat{\Pi}_a^{(s,k)} \hat{\Pi}_b^{(s,k)} + V_s(q, \tau, k). \quad (4)$$

The first term is the phase-free part. The second and third terms form the covariant horizontal dynamics.

2 Relational configuration space and phase bundle

Let $\mathcal{Q}$ be the relational configuration manifold, with local coordinates q^a . The phase is modeled as a $U(1)$ fiber:

$$U(1) \hookrightarrow P \rightarrow \mathcal{Q}. \quad (5)$$

Equivalently, quantum states are sections of a complex line bundle $L_s \rightarrow \mathcal{Q}$:

$$\Psi_s(q, \chi, k) \in \mathcal{H}_s := L^2(\mathcal{Q}, L_s). \quad (6)$$

The phase coordinate $\chi \in U(1)$ is not treated as an isolated coordinate. It is measured covariantly through a phase connection:

$$D_\tau \chi = \dot{\chi} + \mathcal{A}_a^{ABE}(q) \dot{q}^a. \quad (7)$$

Thus phase evolution is not an additive decoration of the shape dynamics. Shape motion and phase motion are connected through the same bundle geometry.

3 Aharonov–Bohm–Berry–Euler connection

Define the total phase connection

$$\mathcal{A}^{ABE} = \mathcal{A}_{AB} + \mathcal{A}_B + \mathcal{A}_E. \quad (8)$$

The components are

$$\mathcal{A}_{AB} = \frac{q_e}{\hbar} A_i(x) dx^i, \quad (9)$$

$$\mathcal{A}_B = i \langle u(q) | du(q) \rangle, \quad (10)$$

$$\mathcal{A}_E = s_E \omega_E. \quad (11)$$

Here $A_i dx^i$ is the electromagnetic vector potential, $\mathcal{A}_B$ is the Berry connection of the local internal/eigenstate frame $|u(q)\rangle$, and ω_E is the Euler/spin connection. In the spin-half sector,

$$s_E = \frac{1}{2}. \quad (12)$$

The holonomy around a closed loop γ is

$$U_\gamma = \exp \left(i \oint_\gamma \mathcal{A}^{ABE} \right), \quad (13)$$

and the accumulated geometric phase is

$$\Phi_{ABE}[\gamma] = \oint_\gamma \mathcal{A}^{ABE} = \Phi_{AB} + \Phi_B + \Phi_E. \quad (14)$$

The curvature of the Berry–Euler part is

$$\mathcal{F}^{BE} = d(\mathcal{A}_B + \mathcal{A}_E) = \mathcal{F}_B + s_E \mathcal{R}_E, \quad (15)$$

where $\mathcal{F}_B = d\mathcal{A}_B$ and $\mathcal{R}_E = d\omega_E$ in the Abelianized sector used here.

4 Information constant, operator domain, and zero-level fluctuations

The project constant is

$$\boxed{\kappa_I = \frac{\ln 2}{24\pi}}. \quad (16)$$

As a mathematical point, the constant alone is not yet a nontrivial dynamical operator. If one writes $\hat{\mathcal{I}} = \kappa_I \mathbf{1}$, then $\hat{\mathcal{I}}$ produces only a global phase in the corresponding evolution. A nontrivial generator requires an operator-valued channel, for instance a winding/intention operator $\hat{W}_s$:

$$\boxed{\hat{\mathcal{I}}_s = \kappa_I \hat{W}_s}. \quad (17)$$

To introduce dynamics from the zero level, write

$$\boxed{\hat{\mathcal{I}}_s(\tau) = \kappa_I \hat{W}_s + \delta \hat{\mathcal{I}}_0(\tau)}. \quad (18)$$

The term $\delta \hat{\mathcal{I}}_0(\tau)$ represents zero-level fluctuations. Formally it may be deterministic, stochastic, semiclassical, or operator-valued. For a unitary Hamiltonian sector the Hermitian part is used:

$$\hat{\mathcal{I}}_{H,s}(\tau) = \frac{1}{2} \left(\hat{\mathcal{I}}_s(\tau) + \hat{\mathcal{I}}_s(\tau)^\dagger \right). \quad (19)$$

4.1 Meaning of a defined domain

The domain of an operator is the set of states on which the operator is legally defined. It is not the numerical value $\ln 2/(24\pi)$. In the present model a minimal domain is

$$\boxed{\text{Dom}(\hat{\mathcal{I}}_s) = \left\{ \Psi_s \in L^2(\mathcal{Q}, L_s) : \hat{W}_s \Psi_s \in L^2(\mathcal{Q}, L_s), \quad \Psi_s \text{ satisfies the holonomy boundary rule} \right\}}. \quad (20)$$

The holonomy boundary rule is

$$\Psi_s(\gamma(1)) = \exp(i\Phi_{ABE}[\gamma]) \Psi_s(\gamma(0)). \quad (21)$$

If spin-half closure is imposed, one may additionally use

$$\Psi(\chi + 2\pi) = -\Psi(\chi), \quad \Psi(\chi + 4\pi) = \Psi(\chi), \quad (22)$$

subject to the chosen spinorial convention.

5 Twin-prime seed and Collatz/Kolac rhythm

For a twin-prime seed $s = (p, p + 2)$, define a scalar seed, for example,

$$n_s = p(p + 2). \quad (23)$$

Then define the Collatz map

$$C(n) = \begin{cases} n/2, & n \equiv 0 \pmod{2}, \\ 3n + 1, & n \equiv 1 \pmod{2}. \end{cases} \quad (24)$$

The Collatz orbit is

$$n_{k+1} = C(n_k), \quad n_0 = n_s. \quad (25)$$

The formalism only requires a positive rhythm functional

$$\rho_s(k) = \rho[n_k, n_{k+1}, \dots] > 0. \quad (26)$$

A bounded reference choice used in the simulations is a parity/log-scale rhythm. Its role is not canonical; it is a numerical placeholder. The formal requirement is that $\rho_s(k)$ clocks the phase generator.

The accumulated intention/information phase over K Collatz steps is

$$\Theta_I(s, K) = \sum_{k=0}^{K-1} \rho_s(k) \langle \hat{\mathcal{I}}_s(k) \rangle. \quad (27)$$

In a fully operator-valued version, the corresponding phase propagator is ordered:

$$U_\Phi(K, 0) = \mathcal{T} \exp \left[-i \sum_{k=0}^{K-1} \rho_s(k) \hat{\mathcal{I}}_s(k) \right]. \quad (28)$$

6 Euler–Berry constraint binding ϵ , D , and phase

The requested Euler–Berry constraint is most cleanly expressed as a closure equation. Let Σ be a spanning surface with boundary $\partial\Sigma = \gamma$. Let

$$\Phi_B[\gamma] = \oint_\gamma \mathcal{A}_B = \int_\Sigma \mathcal{F}_B, \quad (29)$$

and, under the chosen Abelian spin/Euler convention,

$$\Phi_E[\gamma] = \oint_\gamma s_E \omega_E = s_E \int_\Sigma \mathcal{R}_E. \quad (30)$$

The total phase including Aharonov–Bohm and intention is

$$\Phi_{\text{tot}}(s, K, \gamma) = \Phi_{AB}[\gamma] + \Phi_B[\gamma] + \Phi_E[\gamma] + \Theta_I(s, K). \quad (31)$$

Define the topological closure sector $D \in \mathbb{Z}$ and the phase defect $\epsilon_{EB} \in \mathbb{R}$ by

$$\Phi_{\text{tot}}(s, K, \gamma) = 2\pi (D + \epsilon_{EB}). \quad (32)$$

Equivalently,

$$\epsilon_{EB} = \frac{1}{2\pi} \left[\Phi_{AB} + \int_\Sigma (\mathcal{F}_B + s_E \mathcal{R}_E) + \Theta_I(s, K) \right] - D. \quad (33)$$

Exact holonomic closure means

$$\epsilon_{EB} = 0. \quad (34)$$

A practical numerical or empirical tolerance version is

$$|\epsilon_{EB}| \leq \epsilon_\star. \quad (35)$$

6.1 Euler and Berry quantization interpretation

If the Berry curvature belongs to a line bundle with first Chern number

$$c_1(L_B)[\Sigma] = \frac{1}{2\pi} \int_{\Sigma} \mathcal{F}_B \in \mathbb{Z}, \quad (36)$$

and if the Euler curvature obeys a Gauss–Bonnet-type convention

$$\frac{1}{2\pi} \int_{\Sigma} \mathcal{R}_E = \chi(\Sigma), \quad (37)$$

then the Berry–Euler part contributes

$$\frac{1}{2\pi} \int_{\Sigma} (\mathcal{F}_B + s_E \mathcal{R}_E) = c_1(L_B)[\Sigma] + s_E \chi(\Sigma). \quad (38)$$

For $s_E = 1/2$, this contribution naturally carries half-integer structure whenever $\chi(\Sigma)$ is odd. This is the formal location where the Euler/Berry constraint can bind spin-half closure, the phase defect ϵ , and the discrete sector D .

7 Derivation I: canonical phase rotor embedding

Consider the Lagrangian

$$L_s = \frac{1}{2} g_{ab}(q) \dot{q}^a \dot{q}^b + \frac{I_\phi}{2} (D_\tau \chi)^2 + J_{I,s}(\tau, k) D_\tau \chi - V_s(q, \tau, k), \quad (39)$$

where

$$D_\tau \chi = \dot{\chi} + \mathcal{A}_a^{ABE} \dot{q}^a. \quad (40)$$

The intention charge is

$$J_{I,s}(\tau, k) = \hbar \rho_s(k) \hat{\mathcal{I}}_s(\tau, k), \quad (41)$$

or its expectation/scalar reduction in a semiclassical sector.

The canonical momentum conjugate to χ is

$$J = \frac{\partial L_s}{\partial \dot{\chi}} \quad (42)$$

$$= I_\phi D_\tau \chi + J_{I,s}. \quad (43)$$

Thus

$$D_\tau \chi = \frac{J - J_{I,s}}{I_\phi}. \quad (44)$$

The momentum conjugate to q^a is

$$p_a = \frac{\partial L_s}{\partial \dot{q}^a} \quad (45)$$

$$= g_{ab} \dot{q}^b + \frac{\partial}{\partial \dot{q}^a} \left[\frac{I_\phi}{2} (D_\tau \chi)^2 + J_{I,s} D_\tau \chi \right] \quad (46)$$

$$= g_{ab} \dot{q}^b + (I_\phi D_\tau \chi + J_{I,s}) \mathcal{A}_a^{ABE} \quad (47)$$

$$= g_{ab} \dot{q}^b + J \mathcal{A}_a^{ABE}. \quad (48)$$

Therefore the horizontal/covariant momentum is

$$\Pi_a = p_a - J \mathcal{A}_a^{ABE} = g_{ab} \dot{q}^b, \quad (49)$$

and hence

$$\dot{q}^a = g^{ab}\Pi_b. \quad (50)$$

The Hamiltonian is

$$H_s = p_a \dot{q}^a + J\dot{\chi} - L_s. \quad (51)$$

Using $\dot{\chi} = D_\tau \chi - \mathcal{A}_a^{ABE} \dot{q}^a$ and $p_a \dot{q}^a - J\mathcal{A}_a^{ABE} \dot{q}^a = \Pi_a \dot{q}^a$, one obtains

$$H_s = \Pi_a \dot{q}^a + J D_\tau \chi - \frac{1}{2} g_{ab} \dot{q}^a \dot{q}^b - \frac{I_\phi}{2} (D_\tau \chi)^2 - J_{I,s} D_\tau \chi + V_s \quad (52)$$

$$= \frac{1}{2} g^{ab} \Pi_a \Pi_b + (J - J_{I,s}) D_\tau \chi - \frac{I_\phi}{2} (D_\tau \chi)^2 + V_s. \quad (53)$$

Substituting Eq. (44) gives

$$H_s = \frac{(J - J_{I,s})^2}{2I_\phi} + \frac{1}{2} g^{ab} (p_a - J\mathcal{A}_a^{ABE})(p_b - J\mathcal{A}_b^{ABE}) + V_s(q, \tau, k). \quad (54)$$

This is the canonical rotor version of the model.

Its phase-free part is

$$H_\Phi^{\text{rotor}} = \frac{(J - J_{I,s})^2}{2I_\phi}. \quad (55)$$

Its horizontal part is

$$H_\perp = \frac{1}{2} g^{ab} \Pi_a \Pi_b + V_s. \quad (56)$$

8 Derivation II: discrete Floquet phase generator

Because the Collatz/Kolac rhythm is naturally discrete, a Floquet formulation is often cleaner. Define the phase propagator per step by

$$U_\Phi(k+1, k) = \exp \left[-i \rho_s(k) \hat{\mathcal{I}}_s(k) \right]. \quad (57)$$

If

$$U_\Phi(k+1, k) = \exp \left[-\frac{i}{\hbar} \Delta \tau_k H_\Phi(k) \right], \quad (58)$$

then the effective phase Hamiltonian is

$$H_{\Phi, \mathcal{I}, C, s}(k) = \frac{\hbar}{\Delta \tau_k} \rho_s(k) \hat{\mathcal{I}}_s(k). \quad (59)$$

Substituting Eq. (18),

$$H_{\Phi, \mathcal{I}, C, s}(\tau, k) = \frac{\hbar}{\Delta \tau_k} \rho_s(k) \left[\kappa_I \hat{W}_s + \delta \hat{\mathcal{I}}_0(\tau) \right]. \quad (60)$$

This is the form used in Eq. (4).

9 Covariant quantum Hamiltonian

Quantize the phase charge by

$$\hat{J} = -i\hbar \frac{\partial}{\partial \chi}. \quad (61)$$

The horizontal momentum becomes

$$\boxed{\hat{\Pi}_a^{(s,k)} = -i\hbar \nabla_a - \hbar \alpha_s(k) \mathcal{A}_a^{ABE} - \lambda_s(k) \mathcal{I}_a(q).} \quad (62)$$

Here $\alpha_s(k)$ is the effective phase charge/winding sector and $\mathcal{I}_a$ is the geometric trace of the intention channel on $\mathcal{Q}$. In a fixed $\hat{J}$ eigen-sector one may identify $\alpha_s(k)$ with the corresponding phase charge.

The full discrete-time Hamiltonian is

$$\boxed{\hat{H}_s(\tau, k) = \frac{\hbar}{\Delta \tau_k} \rho_s(k) \left[\kappa_I \hat{W}_s + \delta \hat{\mathcal{I}}_0(\tau) \right] + \frac{1}{2} g^{ab} \hat{\Pi}_a^{(s,k)} \hat{\Pi}_b^{(s,k)} + V_s(q, \tau, k).} \quad (63)$$

The same structure in canonical rotor embedding is

$$\boxed{\hat{H}_s^{\text{rotor}}(\tau, k) = \frac{(\hat{J} - \hbar \rho_s(k) \hat{\mathcal{I}}_s(\tau, k))^2}{2I_\phi} + \frac{1}{2} g^{ab} (\hat{p}_a - \hat{J} \mathcal{A}_a^{ABE}) (\hat{p}_b - \hat{J} \mathcal{A}_b^{ABE}) + V_s.} \quad (64)$$

10 Consistency conditions

For the scaffold to be mathematically meaningful, the following conditions are required.

1. **Domain condition:** $\text{Dom}(\hat{\mathcal{I}}_s)$ must be fixed, at least by Eq. (20).
2. **Hermiticity condition:** the Hamiltonian part of $\hat{\mathcal{I}}_s$ must be self-adjoint or replaced by its self-adjoint part.
3. **Rhythm condition:** $\rho_s(k) > 0$ must be defined for every step in the selected Collatz orbit.
4. **Closure condition:** the Euler–Berry defect must obey Eq. (33); exact closure means $\epsilon_{EB} = 0$.
5. **Bundle condition:** the wavefunction must obey the ABE holonomy rule Eq. (21).
6. **No false energy claim:** AB, Berry, and Euler phases are connections/holonomies. They become Hamiltonian terms only after a generator, rotor inertia, or Floquet time step is specified.

11 Reference simulations

The package contains three reference simulations. They are internal consistency checks, not physical evidence.

11.1 Collatz phase accumulation

The script `scripts/collatz_phase_sim.py` uses the twin-prime seed (17, 19), scalar seed $n_s = 323$, and 80 Collatz steps. It computes a bounded Collatz rhythm $\rho_s(k)$, a scalar proxy for $W_s(k)$, and the accumulated intention phase. The resulting closure values are read from `results/collatz_phase_result.json`.

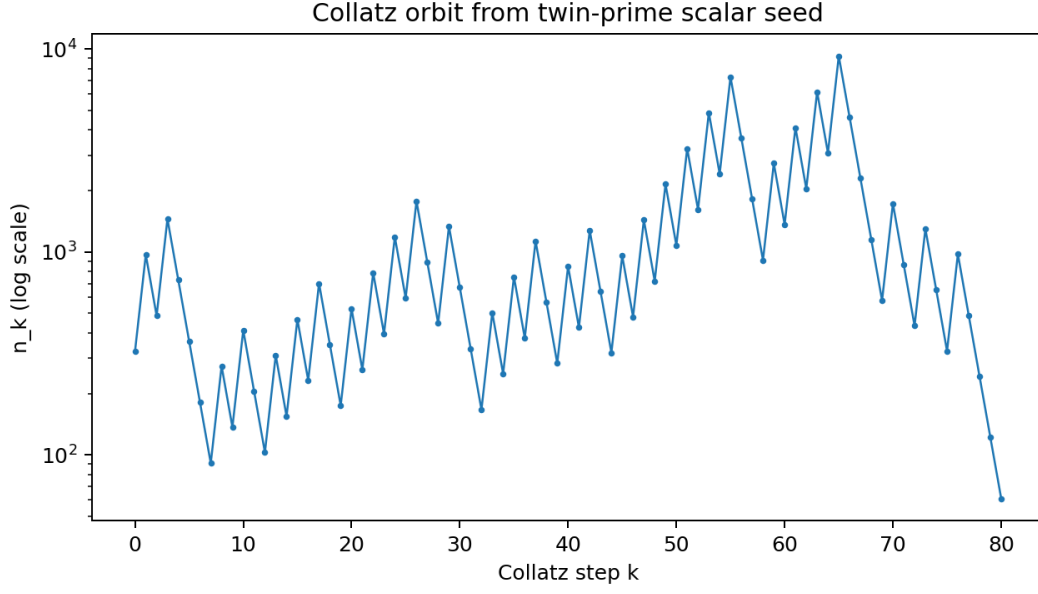


Figure 1: Collatz orbit from the twin-prime scalar seed $17 \cdot 19 = 323$.

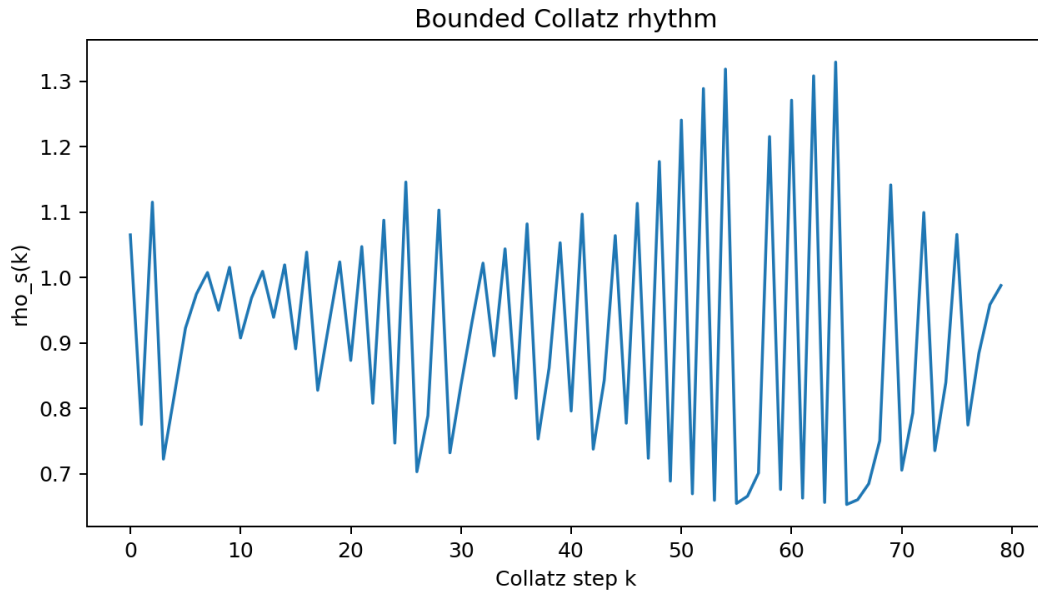


Figure 2: Reference bounded Collatz rhythm $\rho_s(k)$.

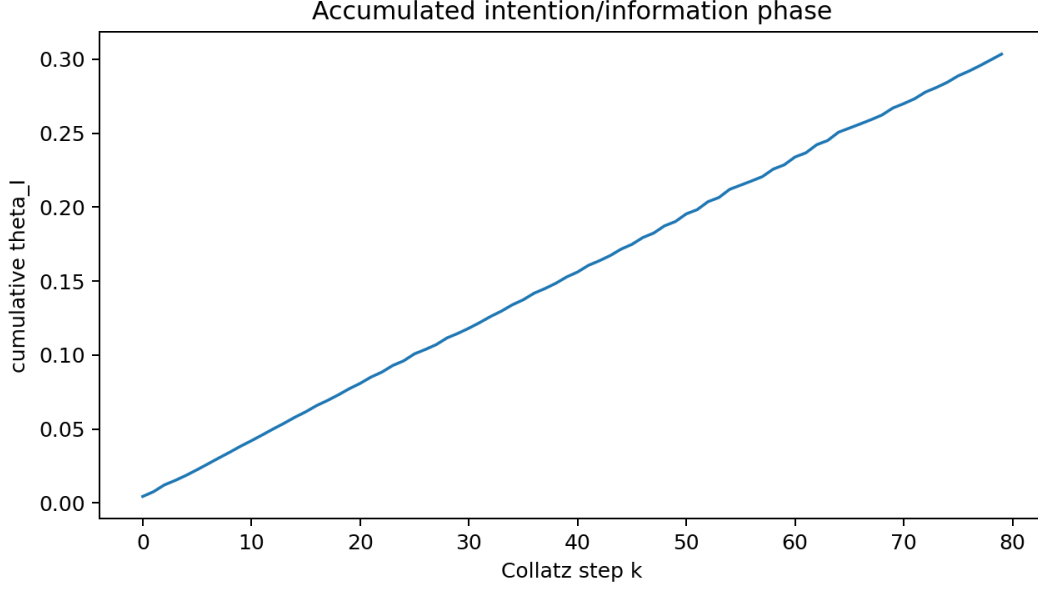


Figure 3: Accumulated scalar intention/information phase Θ_I .

For the default run,

$$\kappa_I = 0.009193150006360484, \quad (65)$$

$$\Theta_I = 0.3033889217620433, \quad (66)$$

$$\Phi_{\text{tot}} = 4.484981575351837, \quad (67)$$

$$D = 1, \quad (68)$$

$$\epsilon = -0.2861930126066793. \quad (69)$$

The unitary modulus error for $e^{i\Phi_{\text{tot}}}$ is zero to displayed precision.

11.2 Euler–Berry closure scan

The script `scripts/euler_berry_constraint_scan.py` scans Berry phase values and Euler characteristics $\chi \in \{-2, -1, 0, 1, 2\}$. It computes

$$\epsilon = \frac{\Phi_{AB} + \Phi_B + s_E 2\pi\chi + \Theta_I}{2\pi} - D. \quad (70)$$

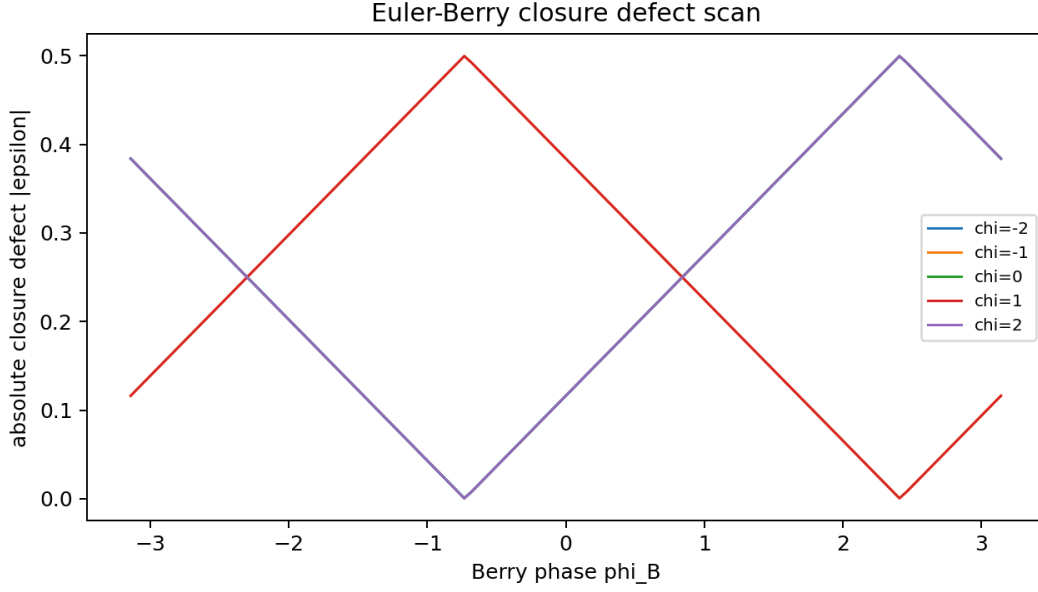


Figure 4: Absolute Euler–Berry closure defect across a small phase/topology grid.

For the default scan, the closest closure point is approximately

$$\chi = -2, \quad \Phi_B = -0.7330382858376185, \quad D = -1, \quad \epsilon \approx -4.8355820958301265 \times 10^{-4}. \quad (71)$$

11.3 Finite matrix Hamiltonian check

The script `scripts/hamiltonian_matrix_demo.py` constructs a finite-dimensional Hermitian matrix model containing a phase term and a nearest-neighbor holonomy coupling. It checks Hermiticity, unitarity of the generated propagator, norm conservation, and energy expectation under a time-independent reference Hamiltonian.

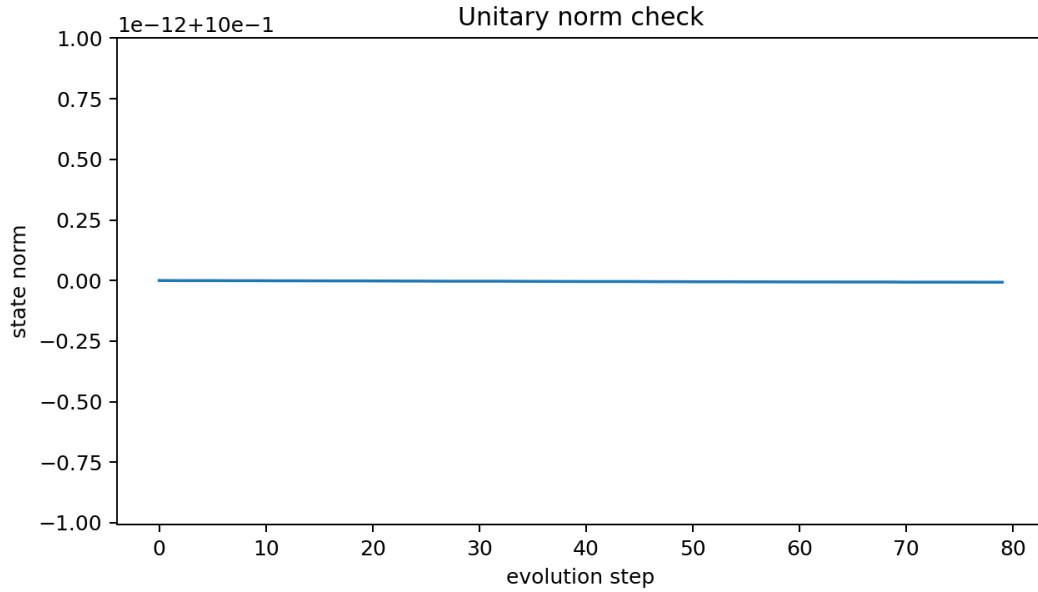


Figure 5: Norm preservation under the finite-dimensional unitary propagator.

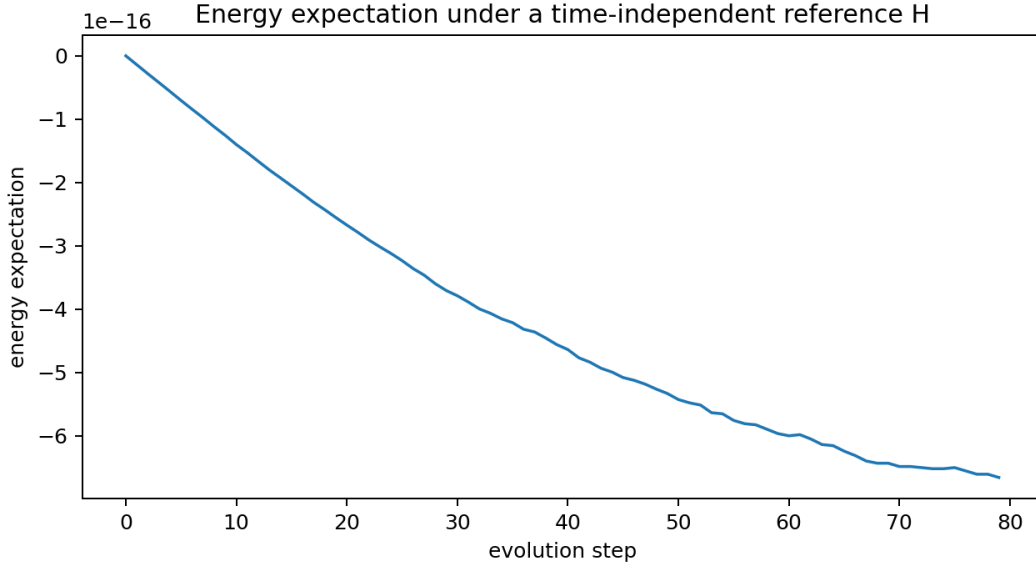


Figure 6: Energy expectation for the time-independent finite matrix reference model.

Default numerical checks:

$$\|H - H^\dagger\| = 0, \quad (72)$$

$$\|U^\dagger U - I\| \approx 6.2610 \times 10^{-15}, \quad (73)$$

$$\max_k \|\psi_k\| - 1 \approx 6.9944 \times 10^{-15}. \quad (74)$$

These values indicate numerical Hermiticity/unitarity in the reference finite model.

12 Interpretation

The formal pipeline is

$$(p, p+2) \longrightarrow n_s \longrightarrow \{n_k\}_{k \geq 0} \longrightarrow \rho_s(k) \longrightarrow \hat{\mathcal{I}}_s(k) \longrightarrow \Phi \longrightarrow \text{ABE holonomic relational dynamics.} \quad (75)$$

The free part is therefore not a free-particle Hamiltonian. It is the free phase/intention generator:

$$H_{\text{free}} = H_{\Phi, \mathcal{I}, C, s}. \quad (76)$$

The ordinary-looking kinetic term belongs to the horizontal geometry and is already covariantly deformed by Aharonov–Bohm, Berry, Euler, and intention-trace connections.

13 Minimal next formal tasks

1. Fix the canonical project definition of D .
2. Choose whether the primary Hamiltonian is the rotor embedding Eq. (54) or the Floquet embedding Eq. (60).
3. Replace scalar $W_s(k)$ in simulations with an explicit finite operator $\hat{W}_s$.
4. Define the fluctuation law for $\delta\hat{\mathcal{I}}_0(\tau)$: deterministic, stochastic, semiclassical, or operator-valued.
5. Define a project-canonical Collatz rhythm $\rho_s(k)$.

6. Add automated tests for Hermiticity, domain admissibility, closure defect, and unitary propagation.

A Source code: Collatz phase simulation

Listing 1: collatz_phase_sim.py

```
#!/usr/bin/env python3
"""
Reference simulation for the intention-phase Hamiltonian scaffold.

This is not an experimental proof. It is a finite numerical model used to
check the internal consistency of the proposed formal structure:

twin-prime seed -> Collatz rhythm -> information/intention phase ->
Euler-Berry closure defect.
"""
from __future__ import annotations

import argparse
import json
import math
from dataclasses import dataclass, asdict
from pathlib import Path
from typing import List, Tuple

import matplotlib.pyplot as plt

KAPPA_I = math.log(2.0) / (24.0 * math.pi)

def is_prime(n: int) -> bool:
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    d = 3
    while d * d <= n:
        if n % d == 0:
            return False
        d += 2
    return True

def collatz(n: int) -> int:
    return n // 2 if n % 2 == 0 else 3 * n + 1

def orbit(n0: int, steps: int) -> List[int]:
    values = [n0]
    n = n0
    for _ in range(steps):
        n = collatz(n)
        values.append(n)
    return values

def bounded_collatz_rhythm(values: List[int], eta: float = 0.35) -> List[float]:
```

```

    """A bounded positive rhythm extracted from a Collatz orbit.

    The exact rhythm map is a model choice. The formal requirement is only
    rho_s(k) > 0. This reference choice uses parity and logarithmic scale so
    that very large Collatz excursions do not dominate the phase numerically.
    """
    logs = [math.log1p(v) for v in values]
    mean = sum(logs) / len(logs)
    var = sum((x - mean) ** 2 for x in logs) / max(1, len(logs) - 1)
    std = math.sqrt(var) if var > 0 else 1.0
    rho = []
    for n, x in zip(values[:-1], logs[:-1]):
        parity = 1.0 if n % 2 else -1.0
        z = math.tanh((x - mean) / std)
        rho.append(1.0 + eta * parity * (0.5 + 0.5 * z))
    return rho

def winding_signal(values: List[int]) -> List[float]:
    """A simple winding channel W_s(k).

    In a full theory W_s is an operator. For the reference scalar simulation
    it is represented by a bounded number derived from the Collatz orbital.
    """
    return [math.atan(math.log1p(n)) / math.pi for n in values[:-1]]

@dataclass
class CollatzPhaseResult:
    twin_prime_seed: Tuple[int, int]
    scalar_seed: int
    steps: int
    kappa_I: float
    eta: float
    theta_intention: float
    phase_ab: float
    phase_berry: float
    phase_euler: float
    total_phase: float
    closest_D: int
    epsilon: float
    unitary_modulus_error: float

def run(seed_p: int, steps: int, eta: float, phase_ab: float, phase_berry: float,
        phase_euler: float) -> CollatzPhaseResult:
    if not (is_prime(seed_p) and is_prime(seed_p + 2)):
        raise ValueError(f"{seed_p} and {seed_p + 2} are not a twin-prime seed")
    n0 = seed_p * (seed_p + 2)
    values = orbit(n0, steps)
    rho = bounded_collatz_rhythm(values, eta=eta)
    W = winding_signal(values)
    increments = [r * KAPPA_I * w for r, w in zip(rho, W)]
    theta_I = sum(increments)
    total = theta_I + phase_ab + phase_berry + phase_euler
    closest_D = round(total / (2.0 * math.pi))
    epsilon = total / (2.0 * math.pi) - closest_D
    U = complex(math.cos(total), math.sin(total))
    unitary_modulus_error = abs(abs(U) - 1.0)

```

```

return CollatzPhaseResult(
    twin_prime_seed=(seed_p, seed_p + 2),
    scalar_seed=n0,
    steps=steps,
    kappa_I=KAPPA_I,
    eta=eta,
    theta_intention=theta_I,
    phase_ab=phase_ab,
    phase_berry=phase_berry,
    phase_euler=phase_euler,
    total_phase=total,
    closest_D=closest_D,
    epsilon=epsilon,
    unitary_modulus_error=unitary_modulus_error,
)

def make_figures(seed_p: int, steps: int, eta: float, out_dir: Path) -> None:
    n0 = seed_p * (seed_p + 2)
    values = orbit(n0, steps)
    rho = bounded_collatz_rhythm(values, eta=eta)
    W = winding_signal(values)
    increments = [r * KAPPA_I * w for r, w in zip(rho, W)]
    cumulative = []
    acc = 0.0
    for x in increments:
        acc += x
        cumulative.append(acc)

    out_dir.mkdir(parents=True, exist_ok=True)

    plt.figure(figsize=(7.2, 4.2))
    plt.plot(range(len(values)), values, marker="o", markersize=2, linewidth=1)
    plt.yscale("log")
    plt.xlabel("Collatz step k")
    plt.ylabel("n_k (log scale)")
    plt.title("Collatz orbit from twin-prime scalar seed")
    plt.tight_layout()
    plt.savefig(out_dir / "collatz_orbit.png", dpi=180)
    plt.close()

    plt.figure(figsize=(7.2, 4.2))
    plt.plot(range(len(rho)), rho, linewidth=1.4)
    plt.xlabel("Collatz step k")
    plt.ylabel("rho_s(k)")
    plt.title("Bounded Collatz rhythm")
    plt.tight_layout()
    plt.savefig(out_dir / "collatz_rhythm.png", dpi=180)
    plt.close()

    plt.figure(figsize=(7.2, 4.2))
    plt.plot(range(len(cumulative)), cumulative, linewidth=1.4)
    plt.xlabel("Collatz step k")
    plt.ylabel("cumulative theta_I")
    plt.title("Accumulated intention/information phase")
    plt.tight_layout()
    plt.savefig(out_dir / "intention_phase_accumulation.png", dpi=180)
    plt.close()

```

```

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--seed-p", type=int, default=17)
    parser.add_argument("--steps", type=int, default=80)
    parser.add_argument("--eta", type=float, default=0.35)
    parser.add_argument("--phase-ab", type=float, default=0.31)
    parser.add_argument("--phase-berry", type=float, default=0.73)
    parser.add_argument("--phase-euler", type=float, default=math.pi)
    parser.add_argument("--out-dir", type=Path, default=Path("results"))
    parser.add_argument("--fig-dir", type=Path, default=Path("figures"))
    args = parser.parse_args()

    args.out_dir.mkdir(parents=True, exist_ok=True)
    result = run(args.seed_p, args.steps, args.eta, args.phase_ab, args.phase_berry,
args.phase_euler)
    (args.out_dir / "collatz_phase_result.json").write_text(json.dumps(asdict(result),
indent=2), encoding="utf-8")
    make_figures(args.seed_p, args.steps, args.eta, args.fig_dir)
    print(json.dumps(asdict(result), indent=2))

if __name__ == "__main__":
    main()

```

B Source code: Euler–Berry constraint scan

Listing 2: euler_berry_constraint_scan.py

```

#!/usr/bin/env python3
"""Scan Euler-Berry phase closure defects across a small parameter grid."""
from __future__ import annotations

import argparse
import json
import math
from pathlib import Path

import matplotlib.pyplot as plt

def closure_defect(phi_ab: float, phi_b: float, chi_euler: float, theta_i: float, spin:
float = 0.5) -> dict:
    # Gauss-Bonnet convention: integral curvature contribution = 2*pi*chi.
    # Spin contribution uses spin * 2*pi*chi.
    phi_e = spin * 2.0 * math.pi * chi_euler
    total = phi_ab + phi_b + phi_e + theta_i
    D = round(total / (2.0 * math.pi))
    epsilon = total / (2.0 * math.pi) - D
    return {"phi_euler": phi_e, "total_phase": total, "D": D, "epsilon": epsilon}

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--out-dir", type=Path, default=Path("results"))
    parser.add_argument("--fig-dir", type=Path, default=Path("figures"))
    args = parser.parse_args()

```



```

args.out_dir.mkdir(parents=True, exist_ok=True)
args.fig_dir.mkdir(parents=True, exist_ok=True)

phi_ab = 0.31
theta_i = 0.42
berry_values = [(-math.pi + 2.0 * math.pi * i / 120.0) for i in range(121)]
chi_values = [-2, -1, 0, 1, 2]
records = []
for chi in chi_values:
    for phi_b in berry_values:
        rec = closure_defect(phi_ab=phi_ab, phi_b=phi_b, chi_euler=chi, theta_i=
theta_i)
        rec.update({"phi_ab": phi_ab, "phi_berry": phi_b, "chi_euler": chi, "
theta_i": theta_i})
        records.append(rec)

(args.out_dir / "euler_berry_scan.json").write_text(json.dumps(records, indent=2),
encoding="utf-8")

plt.figure(figsize=(7.2, 4.2))
for chi in chi_values:
    xs = [r["phi_berry"] for r in records if r["chi_euler"] == chi]
    ys = [abs(r["epsilon"]) for r in records if r["chi_euler"] == chi]
    plt.plot(xs, ys, linewidth=1.2, label=f"chi={chi}")
plt.xlabel("Berry phase phi_B")
plt.ylabel("absolute closure defect |epsilon|")
plt.title("Euler-Berry closure defect scan")
plt.legend(loc="best", fontsize=8)
plt.tight_layout()
plt.savefig(args.fig_dir / "euler_berry_constraint_scan.png", dpi=180)
plt.close()

best = min(records, key=lambda r: abs(r["epsilon"]))
print(json.dumps({"best_closure": best}, indent=2))

if __name__ == "__main__":
    main()

```

C Source code: finite Hamiltonian matrix demo

Listing 3: hamiltonian_matrix_demo.py

```

#!/usr/bin/env python3
"""Finite-dimensional Hermiticity/unitarity check for the phase Hamiltonian."""
from __future__ import annotations

import argparse
import json
import math
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

KAPPA_I = math.log(2.0) / (24.0 * math.pi)

```

```

def build_hamiltonian(n: int, rho: float, delta_tau: float, flux_phase: float) -> np.
    ndarray:
    # Rotor sectors  $m = -M, \dots, +M$ .
    m = np.arange(-(n // 2), n // 2 + 1, dtype=float)
    n = len(m)
    J = np.diag(m)
    W = np.diag(np.tanh(m / max(1.0, n / 4.0)))
    # A simple Hermitian nearest-neighbor shape coupling with a holonomy phase.
    T = np.zeros((n, n), dtype=complex)
    for i in range(n - 1):
        T[i, i + 1] = np.exp(1j * flux_phase)
        T[i + 1, i] = np.exp(-1j * flux_phase)
    H_phase = (rho / delta_tau) * KAPPA_I * W
    H_shape = 0.12 * (J @ J) + 0.05 * T
    return H_phase + H_shape

def unitary_from_hermitian(H: np.ndarray, dt: float) -> np.ndarray:
    eigvals, eigvecs = np.linalg.eigh(H)
    return eigvecs @ np.diag(np.exp(-1j * eigvals * dt)) @ eigvecs.conj().T

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--n", type=int, default=17)
    parser.add_argument("--rho", type=float, default=1.13)
    parser.add_argument("--delta-tau", type=float, default=1.0)
    parser.add_argument("--flux-phase", type=float, default=0.48)
    parser.add_argument("--steps", type=int, default=80)
    parser.add_argument("--out-dir", type=Path, default=Path("results"))
    parser.add_argument("--fig-dir", type=Path, default=Path("figures"))
    args = parser.parse_args()
    args.out_dir.mkdir(parents=True, exist_ok=True)
    args.fig_dir.mkdir(parents=True, exist_ok=True)

    H = build_hamiltonian(args.n, args.rho, args.delta_tau, args.flux_phase)
    hermiticity_error = float(np.linalg.norm(H - H.conj().T))
    U = unitary_from_hermitian(H, dt=0.1)
    unitarity_error = float(np.linalg.norm(U.conj().T @ U - np.eye(U.shape[0])))

    psi = np.zeros(H.shape[0], dtype=complex)
    psi[H.shape[0] // 2] = 1.0
    norms = []
    energies = []
    for _ in range(args.steps):
        norms.append(float(np.linalg.norm(psi)))
        energies.append(float(np.real(psi.conj().T @ H @ psi)))
        psi = U @ psi

    summary = {
        "dimension": int(H.shape[0]),
        "kappa_I": KAPPA_I,
        "hermiticity_error": hermiticity_error,
        "unitarity_error": unitarity_error,
        "initial_energy": energies[0],
        "final_energy": energies[-1],
        "max_norm_error": max(abs(x - 1.0) for x in norms),
    }

```

```

    (args.out_dir / "hamiltonian_matrix_result.json").write_text(json.dumps(summary,
    indent=2), encoding="utf-8")

    plt.figure(figsize=(7.2, 4.2))
    plt.plot(range(len(norms)), norms, linewidth=1.4)
    plt.xlabel("evolution step")
    plt.ylabel("state norm")
    plt.title("Unitary norm check")
    plt.tight_layout()
    plt.savefig(args.fig_dir / "unitary_norm_check.png", dpi=180)
    plt.close()

    plt.figure(figsize=(7.2, 4.2))
    plt.plot(range(len(energies)), energies, linewidth=1.4)
    plt.xlabel("evolution step")
    plt.ylabel("energy expectation")
    plt.title("Energy expectation under a time-independent reference H")
    plt.tight_layout()
    plt.savefig(args.fig_dir / "energy_expectation_demo.png", dpi=180)
    plt.close()

    print(json.dumps(summary, indent=2))

if __name__ == "__main__":
    main()

```

D Source code: run all

Listing 4: run_all.py

```

#!/usr/bin/env python3
from __future__ import annotations

import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
SCRIPTS = ROOT / "scripts"
FIGS = ROOT / "figures"
RESULTS = ROOT / "results"

commands = [
    [sys.executable, str(SCRIPTS / "collatz_phase_sim.py"), "--out-dir", str(RESULTS),
    "--fig-dir", str(FIGS)],
    [sys.executable, str(SCRIPTS / "euler_berry_constraint_scan.py"), "--out-dir", str(
    RESULTS), "--fig-dir", str(FIGS)],
    [sys.executable, str(SCRIPTS / "hamiltonian_matrix_demo.py"), "--out-dir", str(
    RESULTS), "--fig-dir", str(FIGS)],
]

for cmd in commands:
    print("RUN", " ".join(cmd))
    subprocess.check_call(cmd, cwd=str(ROOT))

```